

AtlasFin Take-home

Ship / no-ship decision:

- **(Prioritize, ship)** Min & max value for the credit increase page is adjusted per individual users to minimize the request rejection
 - Max fallback value is \$100 if we can't determine the remaining credit room
- **(Prioritize, ship)** add reasons for why a higher credit increase is not available for the user. Ideally, this link should be a link to an AtlasFin article / shown on the web-app screen directly to increase user retention. For this demo purposes, we're using these publicly available articles.
 - UTILIZATION_T00_HIGH is shown a link to [this article](#)
 - INSUFFICIENT_PAYMENT_HISTORY is shown a link to [this article](#)
 - NEW_CREDIT_LINES is shown a link to [this article](#)
- **(Ship)** UI changes:
 - Add confetti effect if credit increase gets approved to emphasize the good news (and overall make the app feel more interactive)
 - Add a helptip to onboard the user to the "Increase" button if they're seeing it for the first time
 - Match the UI to the existing AtlasFin app on the landing page + AppStore
- **(No ship)** Add more granularity to credit increase amount (increase by \$25 or even by \$1 per step), and allow typing in the credit increase directly, not just the slider.
 - Consideration: UX feels more clunky -> need to run experiments to see whether a more granular credit increase value increases retention rate.

What the next experiment(s) I'd run would be:

- Integration with established FICO score companies (TransUnion, Experian) to track credit score improvements over time.
- Linking to existing credit cards, where AtlasFin becomes an all-in-one platform for credit needs.

Data summarization notes + AI transcript is available on the next few pages.

Recap from summarizing the user & requests data:

- A user is likely to churn if they applied for increase and got rejected
- A user is very likely to stay on the app if the credit limit increase gets accepted
- No difference on the treatment group that doesn't try increasing their credit limit vs the control group
- Getting approved significantly improves retention rate, so we should focus on enabling this feature, while giving resources + try minimizing rejections (since that also significantly reduces user retention rate)

Improving member retention:

- We should also allow users to still go to the increase page, and when they can't get approved, we show them resources to keep them on the platform + showing them pathways on how they can get an increase in the future.

Transcript is also available on the `submission/` folder on GitHub.

```
# React Native app project
_Exported on 9/17/2026 at 18:07:06 CDT from Cursor (3.20.21)_

---

**User**

help me understand this project. i specifically want to work on the react_native_app
directory, not flutter_app

---

**Cursor**

I'll start with the README and project layout, then focus on `react_native_app` so you
can see how that part fits together.

Next I'll read the React Native app, mock API, and design tokens so I can explain how
the credit-increase flow actually works.

This is a two-hour Atlas take-home: decide whether to ship a credit-limit-increase
experiment, then build the next experiment's UX in React Native, not Flutter.

## What you're actually being asked to do

Atlas is a membership product (spending account + card + credit line) for people
traditional banks often miss. They ran an A/B test:

- Treatment can request a credit-line increase in the app
- Control never sees an Increase button and never gets an increase
- Requests are a soft inquiry (no credit-score hit)

Your job, weighted:

| Weight | Axis |
|---|---|
| 25% | Read the experiment data in `data/` |
| 35% | Prioritize (ship / no-ship / something else) |
| 40% | User experience of the next experiment |

Deliverables:
```

1. One-page `DECISIONS.pdf` – ship/no-ship + evidence + next experiment
2. Working UX for that next experiment's groups
3. Full AI transcript if you used AI

The north star they give you is **member retention**. The app work is only half of it; the CSVs matter as much as the screens.

Repo layout (what to ignore vs use)

```
...
atlasfin-takehome/
  README.md           # assignment
  data/               # experiment CSVs – join on user_id
  design/tokens.json  # shared design tokens
  mock-api/           # local credit-line API on :4010
  react_native_app/   # ← you work here
  flutter_app/        # ignore
...
```

The RN and Flutter apps are the **same baseline treatment flow**, duplicated. Tokens are mirrored in all three places – if you change colors/spacing/type, change `design/tokens.json` and `react\_native\_app/src/tokens.ts` together.

The React Native app

It's a small **Expo** app (React Native 0.86 + Expo 57). Almost all product code is one screen file plus two helpers:

File	Role
`App.tsx`	Entire UI: Home → Request → Decision
`src/apiClient.ts`	HTTP to mock-api, plus persona switching
`src/tokens.ts`	Colors, spacing, type (use these; they say don't spend time on visual design)
`index.ts`	Expo entry
`app.json`	Expo config

No React Navigation. Screens are a tagged union in state:

```
```17:20:react_native_app/App.tsx
type Screen =
```

```
| { name: 'home'; refreshKey: number }
| { name: 'request'; limitCents: number }
| { name: 'result'; result: ApiResult | null; error: string | null };
...
```

### ### The three screens

#### \*\*1. Your credit line (`HomePage`)\*\*

Loads `GET /v1/credit-line`. Shows limit and spent. **\*\*Increase\*\*** is hidden once `increase_request` is non-null (one request per member). A footer **\*\*PERSONA\*\*** picker switches the mock member via `X-Test-Persona`.

#### \*\*2. Request an increase (`RequestPage`)\*\*

Slider from **\*\*\$100-\$2,500 in \$50 steps\*\***. Submits the **\*\*increase amount\*\*** (not the target limit) as `requested_amount_cents`.

#### \*\*3. Decision (`ResultPage`)\*\*

Approved → "Approved" + new limit. Declined → "Declined" + generic "We can't increase your limit right now." That generic decline copy is a known gap: the API returns specific reason codes that compliance wants shown.

### ### How the API client works

`ApiClient` talks to `http://localhost:4010` (Android emulator uses `10.0.2.2`). It always sends `x-test-persona`. Two methods:

- `getCreditLine()` → `GET /v1/credit-line`
- `submitIncreaseRequest(cents)` → `POST /v1/credit-line/increase-requests`

Persona lives on the client instance, so switching it on Home refetches that member's account.

### ## Mock API (run this first)

`cd mock-api && npm start` → `http://localhost:4010`. In-memory, resets on restart.

Every persona has a **\*\*fixed\*\*** `max_eligible_limit_cents`. Approval is just:

`current_limit + requested_increase ≤ max_eligible_limit`

The API will still accept an overshoot; it returns `201` with `DECLINED` + a reason, not a validation error. Putting a member in a doomed slider range is **the app's decision**.

Persona	Limit	Ceiling	Room	If they overshoot
---	---	---	---	---
`approved`	\$2,500	\$5,000	\$2,500	always approved in slider range
`utilization-too-high`	\$1,000	\$1,800	\$800	`UTILIZATION_TOO_HIGH`
`new-credit-lines`	\$500	\$1,000	\$500	`NEW_CREDIT_LINES`
`insufficient-history`	\$1,500	\$1,500	<b>none</b>	`INSUFFICIENT_PAYMENT_HISTORY`

for any amount

Required decline copy (from `mock-api/README.md`):

- `UTILIZATION\_TOO\_HIGH` - high utilization on this card and/or others
- `INSUFFICIENT\_PAYMENT\_HISTORY` - need a longer run of on-time payments
- `NEW\_CREDIT\_LINES` - too many new credit lines recently

Other rules: one request per member (`409 ALREADY\_REQUESTED`); approved increases are live immediately (`limit\_cents` updates).

## ## Experiment data (the other half)

Join `data/users.csv` ↔ `data/requests.csv` on `user\_id`.

- ~1M users, ~475k requests
- Randomized ~50/50 treatment vs control at signup
- Members from early June 2026; snapshot **2026-08-25**
- `max\_eligible\_limit\_cents` is **fixed at join** - don't propose re-underwriting it
- Retention proxy: `account\_closed\_ts` blank = still open

`requested\_limit\_cents` in the CSV is the **target total limit**, not the increase. The live API is the opposite: it takes the **increase**.

## ## How to run the RN app

```
```bash
# terminal 1
cd mock-api && npm start

# terminal 2
cd react_native_app
```

```
npm install
npm run web          # easiest
# or: npm run ios / npm run android
...
```

The README says don't spend time on visual design – use ``src/tokens.ts`` and ship product decisions.

What's interesting to change in RN

The baseline is a thin treatment prototype. Likely product/UX issues you'll want to decide on after looking at the data:

1. **Decline copy** is generic; compliance wants specific reasons.
2. **Slider can request more than the member's room** – API allows it, app currently doesn't cap it. ``insufficient-history`` has **zero** room and still sees Increase.
3. **Home is sparse** – no available credit, utilization, months-on-book, or remaining room (those are derivable from the API).
4. **No control-group UI** – you'll need that if the next experiment has groups.
5. **One-shot request** – after submit, Increase disappears; declined members get no path forward.

The highest-leverage RN work is probably: read the CSVs, pick a next experiment aimed at retention, then implement those experiment groups in ``react_native_app/`` against the existing mock API.

If you want a next step, I can either run a first pass on the experiment data or sketch what to change in the RN screens.

User

just to clarify, is this assignment strictly to improve the UI + adding features that will increase user retention?

Cursor

No. Retention is the **goal of the decision**, not a request to polish the current screens and add features.

The assignment is a **product judgment** on the experiment that already ran, then a **next experiment** you can actually demo.

1. **Read the data** (25%) – did treatment retain better than control? Who requested, who got declined, who churned?
2. **Prioritize** (35%) – ship the current Increase flow, kill it, or do something else. Then say what you'd test next, and why that helps **member retention %**.
3. **UX** (40%) – fully working designs for **that next experiment's groups**, not generic UI improvements.

The RN app is a baseline of the treatment that already shipped to some members. You are not being asked to "make Home nicer." You are being asked to decide what should happen *after* this experiment, then build those variants.

So: if the data says the Increase button hurts retention for declined members, the right UX might be hiding it, capping the slider, or showing a different path – not adding more features. If it helps, you might ship it and test a follow-up. The PDF is the main artifact; the app exists to make the next experiment concrete.

User

what is the best way to ingest both requests and users.csv so i can make the user retention decision based on the experiment results?

Cursor

I'll inspect the CSV headers and a few rows first, then recommend the fastest way to join them and compute retention.

DuckDB over the CSVs in place is the best ingest for this. Don't load them into Postgres, Excel, or pandas unless you already prefer Python.

You have ~1M users and ~475k requests. That's small for DuckDB and painful for spreadsheets. You need a **left join** (keep every user, including control who never requested) and **tenure-aware retention**, not a raw closed-account %.

Setup

```
```bash
```

```
pip install duckdb
```

```
```
```

Then query the files directly – no import step:

```
```python
```

```
import duckdb
```

```
con = duckdb.connect()
```

```
con.execute("""
```

```
CREATE VIEW users AS
```

```
SELECT
```

```
 user_id,
```

```
 joined_ts::DATE AS joined_date,
```

```
 experiment_group,
```

```
 NULLIF(account_closed_ts, '')::DATE AS closed_date,
```

```
 starting_limit_cents,
```

```
 max_eligible_limit_cents
```

```
FROM read_csv_auto('data/users.csv');
```

```
CREATE VIEW requests AS
```

```
SELECT
```

```
 user_id,
```

```
 decision,
```

```
 NULLIF(decline_reason, '') AS decline_reason,
```

```
 requested_limit_cents,
```

```
 requested_ts::TIMESTAMP AS requested_ts
```

```
FROM read_csv_auto('data/requests.csv');
```

```
""")
```

```
```
```

Blank `account\_closed\_ts` means still open at the ****2026-08-25**** snapshot. `NULLIF` turns those blanks into real nulls.

The table you actually analyze

```
```python
```

```
con.execute("""
```

```
CREATE VIEW members AS
```

```

SELECT
 u.*,
 r.decision,
 r.decline_reason,
 r.requested_limit_cents,
 r.requested_ts,
 r.user_id IS NOT NULL AS requested,
 u.closed_date IS NULL AS retained,
 DATE '2026-08-25' AS snapshot_date,
 datediff('day', u.joined_date, DATE '2026-08-25') AS days_observed,
 datediff('day', u.joined_date, u.closed_date) AS days_to_close
FROM users u
LEFT JOIN requests r USING (user_id)
""")
...

```

That join is the whole ingest. Control has no request rows; if you inner-join, you throw away the experiment.

## ## What to compute first (ITT)

Assignment was randomized, so the retention decision is **intent-to-treat**, not "people who requested vs people who didn't."

```

```sql
SELECT experiment_group,
       count(*) AS n,
       avg(retained::INT) AS retained_pct,
       avg((closed_date IS NOT NULL)::INT) AS closed_pct
FROM members
GROUP BY 1;
...

```

Do **not** stop at that number. Later joiners have less time to churn. Restrict to people with enough follow-up, e.g. closed within 30/45/60 days among those observed at least that long:

```

```sql
SELECT experiment_group,
 avg((days_to_close <= 30)::INT) AS closed_within_30d
FROM members
WHERE days_observed >= 30

```

```
GROUP BY 1;
...
```

Same idea by join week, starting limit, and (for treatment only) approved / declined / never requested.

## ## Pitfalls that will wreck the decision

1. **Selection bias.** Requesting is not random. Compare treatment vs control first. Then look at requested vs not *inside treatment* as a diagnostic, not as the experiment result.
2. **requested\_limit\_cents` is the target limit**, not the increase. Headroom is ``max_eligible_limit_cents - starting_limit_cents``. Asking above the ceiling is a decline by construction.
3. **Control should have zero requests.** If they don't, the randomization story is broken – check that immediately.
4. **One row per user in requests?** Confirm with ``count(*)`` vs ``count(DISTINCT user_id)`` before joining 1:1.
5. Pandas is fine (~1M rows, maybe 100-200MB RAM) if you already think in Python. Excel/Sheets is not.

If you want, I can run this ingest and pull the ITT retention numbers next.

---

## **\*\*User\*\***

is it better to run it over jupyter / using python? i have more experience in it rather than duckdb

---

## **\*\*Cursor\*\***

Yes. Use Python. For a two-hour take-home, the tool you already think in is the better one.

DuckDB is faster to *query*, not faster for *you* if you'd be learning SQL-on-CSV under a timebox. Pandas will eat 1M users + 475k requests in a few seconds and a couple hundred MB of RAM.

**\*\*Jupyter vs a `.py` script:\*\*** either works. Jupyter is nicer for poking at tables and copying numbers into the PDF. A script is nicer if you already know what you want. Don't spend time setting up a notebook if a REPL or ``python analysis.py`` is faster for you.

Minimal ingest:

```
```python
import pandas as pd

SNAPSHOT = pd.Timestamp("2026-08-25")

users = pd.read_csv(
    "data/users.csv",
    parse_dates=["joined_ts", "account_closed_ts"],
)
requests = pd.read_csv(
    "data/requests.csv",
    parse_dates=["requested_ts"],
)

df = users.merge(requests, on="user_id", how="left")
df["retained"] = df["account_closed_ts"].isna()
df["days_observed"] = (SNAPSHOT - df["joined_ts"]).dt.days
df["days_to_close"] = (df["account_closed_ts"] - df["joined_ts"]).dt.days
```
```

Then ITT is just:

```
```python
df.groupby("experiment_group")["retained"].mean()
```
```

Same rules as before: left join, treatment vs control first, then tenure windows (``days_observed >= 30`` and closed within 30 days). Don't inner-join or you'll drop control.

Skip DuckDB unless you hit a pandas wall – you won't at this size.

---

**\*\*User\*\***

lets do that. I'd like you to output a visualization and export it as pdf. create a new directory called data-summary/ and store both the python script and the pdf export there.

---

**\*\*Cursor\*\***

I'll ingest both CSVs in Python, build a retention-focused summary visualization, and write the script plus PDF into `data-summary/`.

I'll put a local virtualenv in `data-summary/` so we can install pandas without touching system Python, then generate the PDF.

The 1M-row `apply` would be slow – I'll vectorize that path, then run the script and generate the PDF.

Page 2 clips the path labels, and the overshoot chart duplicates the decline split. I'll fix the layout, swap in a 14-day churn chart, and regenerate the PDF.

Dollar signs were parsed as math text and a notes line was clipped. I'll escape those, tighten the copy, and regenerate.

Both CSVs are ingested in pandas and summarized in `data-summary/`.

- Script: `data-summary/analyze\_retention.py`
- PDF: `data-summary/retention\_summary.pdf`

Rerun with:

```
```bash
data-summary/.venv/bin/python data-summary/analyze_retention.py
```
```

The PDF is two landscape pages: ITT + tenure on page 1, treatment paths on page 2.

**\*\*What the numbers say\*\***

Naive ITT looks like a wash that slightly hurts retention: treatment closed **\*\*12.4%\*\*** vs control **\*\*12.1%\*\*** (+0.31 pp,  $p \approx 2 \times 10^{-6}$ ). That headline is misleading.

- **First 14 days:** treatment closed **12.0%** vs control **6.0%** (+6 pp). Almost all of that spike is declines (**31.7%** closed within 14 days). Approved members match control at **6.0%**.
- **Longer follow-up:** by 45-60 days, treatment is **better** (-0.9 to -1.5 pp). Early join-week cohorts show the same pattern.
- **95%** of treatment requested. **Every decline is an ask above max eligible**; every approval is in-range. Decline *reason* doesn't matter – all three reasons churn at ~32%.
- **23%** of members have zero headroom at signup and still request at 95%. The current slider lets them do that.

So the retention decision is not “ship vs kill the Increase button.” It's: approvals look protective, declines look immediately harmful, and the app currently puts people with no room on a doomed path.

---

**User**

what is ITT again?

---

**Cursor**

**ITT** is **intent-to-treat**: compare people by the group they were **randomly assigned to**, not by what they actually did.

Here that means:

- **Treatment** vs **control**
- Not approved vs declined
- Not requested vs didn't request

Assignment was randomized at signup, so that comparison is the fair experiment result. Requesting, getting approved, or getting declined is **chosen** (and selected), so those splits are useful for diagnosing *why* retention moved, but they are not the experiment.

In this data, ITT is “did offering the Increase flow change closed-account rate?” – currently **12.4% treatment vs 12.1% control**.

### **\*\*User\*\***

with the current implementation, how is it determining whether the user increase request gets rejected? does it only get evaluated once the user submits the increase request, or is there a way for the user to find out what their max increase request value is before submitting the request?

### **\*\*Cursor\*\***

I'll check the app and mock API to confirm when approval is decided and whether max eligible is shown before submit.

Rejection is decided **\*\*only when they submit\*\***. The app never shows remaining room or a max ask beforehand.

### **\*\*How the server decides\*\***

On ``POST /v1/credit-line/increase-requests``, the mock API does:

``current limit + requested increase ≤ max_eligible_limit`` → ``APPROVED``, else ``DECLINED`` plus a persona reason code.

That is the whole underwriting rule. An overshoot is a **\*\*201 decision\*\***, not a validation error. The slider bounds (\$100-\$2,500 in \$50 steps) are separate and only reject with ``422`` if the amount is outside those steps.

### **\*\*What the member sees before submit\*\***

Home loads ``GET /v1/credit-line``, which **\*\*does\*\*** return ``max_eligible_limit_cents``. Remaining room is just ``max_eligible_limit_cents - limit_cents``. The React Native app **\*\*ignores that field\*\***. Home only shows credit limit and spent. Increase is hidden only after a request already exists.

The request screen always uses a **\*\*\$100-\$2,500\*\*** slider. It does not cap to remaining room. Copy is "New limit **\*\*if approved\*\***," then they find out on the result screen.

So today a member with \$0 room (``insufficient-history``) or \$800 room (``utilization-too-high``) can still drag to \$2,500 and only learns they were declined after burning their one request. The API would let the app show or cap the max in advance; the current UI chooses not to.

---

#### **\*\*User\*\***

great. with that in mind, here are the 2 things we should prioritize:

1. The slider min & max value should adapt to the remaining room for the credit increase for the given user/persona
2. in the case that the request is declined, add a reason for decline, as well as a hyperlink to a public resource on how to fix it (feel free to use a basic article link as a placeholder)
3. in the case that the request is accepted, improve the UI to emphasize on the user's ecstatic feeling. Something like a confetti popping effect on the screen would be nice.

---

#### **\*\*Cursor\*\***

I'll adapt the slider to remaining room, add decline reasons with a help link, and celebrate approvals with confetti. Then I'll verify the flow in the browser.

I'll update the request flow so the slider stays within remaining room, declined results include a reason and help link, and approvals get a celebration with confetti. Because capping the slider prevents overshoot, I'll also handle zero-room members so they aren't sent into a doomed ask.

The mock API still has prior requests stored, so I'll reset it and then walk through the new flows in the browser.

Confetti likely never ran on web because of the native driver. I'll switch that off, keep the pieces on screen longer, then re-check approval, the adaptive slider, and the decline path.

All three are in ``react_native_app/App.tsx``.

#### **\*\*1. Slider follows remaining room\*\***

Room is ``max_eligible_limit_cents - limit_cents``, then clipped to the API's \$100-\$2,500 / \$50 steps. Home also shows "Room to increase."

- ``approved`` → \$100-\$2,500
- ``utilization-too-high`` → \$100-\$800
- ``new-credit-lines`` → \$100-\$500
- ``insufficient-history`` → \$0 room, so **\*\*Increase is hidden\*\*** instead of a doomed slider. They get an explanation plus a CFPB link.

## **\*\*2. Declined requests\*\***

The result screen uses the API ``reason_codes`` (compliance copy from the mock API) and a CFPB article for that reason. I opened the new-credit-lines link; it goes to the public CFPB inquiry article.

## **\*\*3. Approvals\*\***

Approved results now use the positive green treatment: "You're approved!", the new limit, and a confetti burst.

Because the slider can no longer overshoot, the mock will mostly **\*\*approve\*\*** people who still have room. That's the retention-minded change: don't let someone ask for more than they can get. Decline UI is still there when the API returns ``DECLINED``.

Restart ``mock-api`` if you want to walk the request flow again — each persona can only request once until the server resets.

---

## **\*\*User\*\***

now that i tested it, this current flow will never really hit a "Decline", since the max increase request will always follow the remaining room for the given persona. The better UI here would be to add a simple link ("What if I would like to increase my credit limit even more?" or something along that line) which can then show them the appropriate resource that matches why they would be declined for the higher amount respectively.

Also, in the case where room is \$0, don't show the warning on the homepage directly. Still show the "Increase" button and show the error there

---

## **\*\*Cursor\*\***

I'll move the \$0-room message off Home onto the request screen, keep Increase visible, and add a "want a higher limit?" link that shows the matching help resource.

I'll keep Increase on Home for \$0-room members, move that message to the request screen, and add a "want even more?" link with the matching help resource.

Two flow changes are in:

**\*\*Home no longer warns when room is \$0.\*\*** Increase stays on the home screen. ``insufficient-history`` still shows the account card (including \$0 room), and the ineligibility message lives on the request screen: reason plus a CFPB payment-history article. No submit.

**\*\*Asking for more than remaining room doesn't take a doomed request.\*\*** Under the slider there's *\*What if I would like to increase my credit limit even more?\** Tapping it shows why a higher ask would be declined for that member, plus the matching article:

- high utilization → utilization CFPB article
- new credit lines → inquiries article
- no remaining room → payment history article

The slider is still capped, so submitting stays inside remaining room and can still approve. The extra link is how they learn what would block a larger increase, without burning the one-shot request.

---

**\*\*User\*\***

great. for the rejection reasons, give me the article link you used for each scenario

---

**\*\*Cursor\*\***

All three are CFPB Ask CFPB articles:

**\*\*High utilization\*\*** (``UTILIZATION_TOO_HIGH`` - ``utilization-too-high`` persona)  
<https://www.consumerfinance.gov/ask-cfpb/how-does-my-credit-card-use-affect-my-credit-score-en-1883/>

**\*\*Insufficient payment history\*\*** (`INSUFFICIENT\_PAYMENT\_HISTORY` -  
`insufficient-history` persona, including \$0 room)  
<https://www.consumerfinance.gov/ask-cfpb/how-do-i-get-and-keep-a-good-credit-score-en-318/>

**\*\*New credit lines\*\*** (`NEW\_CREDIT\_LINES` - `new-credit-lines` persona)  
<https://www.consumerfinance.gov/ask-cfpb/how-do-credit-inquiries-affect-my-credit-report-en-1317/>

`approved` uses the utilization article if they ask about going above remaining room, since that's the reason the mock would attach to an overshoot for that persona.

---

#### **\*\*User\*\***

for a user that sees the "Increase" button for the first time, show a helptip that points them to "What's new".

Also, for a user that has the credit room, show an intermediary screen that shows an overlay that says something like "Congratulations, due to your <reason>, you are eligible to request a credit line increase!"

---

#### **\*\*Cursor\*\***

I'll add a first-time "What's new" helptip on Increase, and an eligibility congratulations overlay for members who still have room. Then I'll verify both in the browser.

---

#### **\*\*User\*\***

the button should be associated to the "Increase" button directly, not under a "What's new" pill.

---

#### **\*\*Cursor\*\***

I'll attach the "What's new" helptip to the Increase button and remove the separate pill.

The pill is gone. The first-time **What's new** helptip now sits under **Increase** and points at that button. **Got it** still dismisses it.

---

**User**

the congratulations page shouldn't be a popup on the homepage. it should be an overlay that covers the increase page (blur), which then disappears after the user acknowledges it

---

**Cursor**

I'll move the congratulations overlay onto the request screen as a full-page blur that dismisses after they acknowledge it.

The congratulations moment now lives on the **increase** screen, not Home.

Tapping **Increase** goes to the request page. If they have remaining room, a blur overlay covers that page (slider visible underneath). **Continue** dismisses it and they can set the amount.

Members with \$0 room still skip the overlay and see the ineligibility message on the request screen.

---

**User**

@react\_native\_app/App.tsx:88 max doesn't need to be capped at \$2,500 if their room allows for more. For error fallback (if room is not defined), we can set the limit to \$1,000

---

**Cursor**

I'll drop the \$2,500 slider cap so max follows remaining room, and use \$1,000 only when room isn't defined.

The slider max now follows remaining room, snapped to \$50 steps, with no \$2,500 ceiling.

If ``max_eligible_limit_cents`` is missing, the max falls back to **\*\*\$1,000\*\***. Zero remaining room is still treated as ineligible, not as that fallback.

---

**\*\*User\*\***

create a commit message and commit title following the 50/72 rule describing the committed changes

---

**\*\*Cursor\*\***

Committed on ``feat/edward-submission`` as ``11d8a52``.

**\*\*Title\*\*** (39/50)

Reduce doomed credit-increase requests

**\*\*Body\*\*** (wrapped at 72)

Cap the slider to remaining room, explain why a larger ask would fail, and celebrate approvals. Add a first-time Increase helptip, an eligibility overlay, and a retention summary from the experiment CSVs.

---

**\*\*User\*\***

now I'd like you to match the UI / UX of this dashboard so that it can be integrated with the primary AtlasFin app. (Only UI and design changes, no need to add other mock data for the dashboard). the idea here is that we can directly plug this new interface to the existing app and it will look just right in place

---

**\*\*Cursor\*\***

I'll restyle the React Native screens to match the Atlas dashboard: purple brand, white cards, and the same button/type treatment—design only. Then I'll check it in the browser.

I'll restyle tokens and screens to Atlas's purple, pill buttons, and white cards—keeping the same data and flows.

The styles block got truncated when SecondaryButton was updated. I'll restore the closing tags and rewrite the stylesheet.

I'll finish the Atlas restyle: restore the broken stylesheet, then align the home card with the production dashboard layout.

---

**\*\*User\*\***

here's another screenshot that's relevant

---

**\*\*Cursor\*\***

The home widget in this shot is the plug-in point — I'll match that Credit Card card and circular **\*\*Increase Limit\*\*** action, and finish the restyle.

---

**\*\*User\*\***

im not seeing any changes yet

---

**\*\*Cursor\*\***

The bundler likely stuck on an earlier syntax error. I'll check Metro, reload the app, and confirm the new UI is actually on screen.

The new UI is on web after a reload — Metro had been stuck on a broken bundle. I'll tighten the layout to a phone-width card and check the increase flow.

---

**\*\*User\*\***

tooltip needs to be shown on the "Increase Limit" button

---

**\*\*Cursor\*\***

I'll attach the helptip directly to the Increase Limit control so it points at that button, not the card.

The first-time helptip now sits on the **\*\*Increase Limit\*\*** action: the caret points at the circle, and the bubble is inside the credit card rather than floating below it. Reload if you still have the old layout cached.

---

**\*\*User\*\***

where did oyu get the "pending" number from?

---

**\*\*Cursor\*\***

It comes from the mock API's existing ``pending_holds_cents`` field on ``GET /v1/credit-line`` — not a new number I invented.

For the ``approved`` persona that's ``6550`` cents, which is the **\*\*\$65.50\*\*** on the card. Available to spend is ``limit_cents - balance_cents - pending_holds_cents``.

I surfaced it so the home card could match Atlas's Current balance / Pending rows. If you'd rather not show pending, I can drop that row.

---  
**\*\*User\*\***

great. now summarize what we did in this chat session

---  
**\*\*Cursor\*\***

This session was **\*\*design-only\*\***: restyle the credit-line increase flow so it can sit in the real Atlas app, without adding extra dashboard mock data.

**\*\*Visual system\*\***

- Tokens now follow Atlas: purple ``#5B4CFF``, navy ``#1A1464``, gray paper, white cards, 20px card radius, pill buttons.
- Home is the Atlas credit-card widget: purple header, white card, large **\*\*Available to spend\*\***, Current balance / Pending rows, circular **\*\*Increase Limit\*\***.
- Request, eligibility overlay, and result screens use the same purple pills, white cards, and type.

**\*\*What we used vs invented\*\***

- Pending is ``pending_holds_cents`` from ``GET /v1/credit-line`` (e.g. \$65.50 on ``approved``).
- Available is ``limit - balance - pending``.
- No fake last-4, cashback tiles, or "Hello Alden".

**\*\*UX wiring\*\***

- First-time **\*\*What's new\*\*** helptip is attached to **\*\*Increase Limit\*\***, not floating under the card.
- Existing experiment behavior is unchanged: capped slider, decline guidance, eligibility overlay, confetti on approval.

**\*\*Why it looked like nothing changed\*\***

- Metro had crashed on a broken stylesheet mid-edit, so the simulator/web kept the old teal UI until a reload.